Implement Symbol Table Generation:

Practical - 3

Code :

```
%{
#include <stdio.h>
#include <string.h>

int count = 0;

struct SymbolTable
{
   char name[50];
} table[100];

int search(char *id)
{
   for(int i = 0; i < count; i++)
   {
      if(strcmp(table[i].name, id) == 0)
         return 1;
   }
   return 0;
}
%}

%%
int|float|char|double|long|short      { }

[a-zA-Z_][a-zA-Z0-9_]* {
   if(!search(yytext))
   {
```

```
        strcpy(table[count].name, yytext);
        count++;
    }
}

[0-9]+      { }

[ \t\n]+    { }

.           { }
%%

int main()
{
    printf("Enter the program:\n");
    yylex();

    printf("\nSYMBOL TABLE \n");
    printf("Index\tIdentifier\n");

    for(int i = 0; i < count; i++)
    {
        printf("%d\t%s\n", i + 1, table[i].name);
    }

    return 0;
}

int yywrap()
{
    return 1;
}
```

Input.txt

```
int a;
```

```
int b;
float c;
char ch;
double total;

a = 10;
b = 20;
c = 15;
ch = 'A';

total = a + b + c;

int result;
result = total;
```

Output :

```
(base) shivampatel@Shivams-MacBook-Pro ~ % gcc lex.yy.c
(base) shivampatel@Shivams-MacBook-Pro ~ % ./a.out < input2.txt
Enter the program:

SYMBOL TABLE
Index   Identifier
1       a
2       b
3       c
4       ch
5       total
6       result
(base) shivampatel@Shivams-MacBook-Pro ~ %
```